МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1

по курсу «Компьютерная графика»

Основы 3D графики

Вариант 11 — Правильный икосаэдр

Выполнил: Пекин Семён
Группа: М8О-305БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы — построить простой трёхмерный объект и изучить его отображение на плоскость экрана с помощью матриц проекции и аффинных преобразований. По варианту 11 необходимо построить правильный икосаэдр. Программа должна работать в реальном времени и позволять изменять параметры объекта через интерфейс.

Для реализации использованы C++, Vulkan, GLFW и ImGui. В программе выполнены дополнительные задания 1–5 (на оценку «4»):

1. Переключение между перспективной и ортографической проекциями.
2. Изменение положения, поворота и масштаба фигуры.
3. Движение по круговой траектории и вращение с возможностью паузы, изменения скорости и радиуса.
4. Изменение цвета через интерфейс.
5. Процедурное задание цветов вершин и умножение на выбранный цвет.

Построение икосаэдра

Правильный икосаэдр состоит из 12 вершин и 20 равносторонних треугольных граней. Фигура задана в локальной системе координат, её центр находится в начале координат.

Координаты вершин задаются с использованием золотого сечения $t = (1 + \sqrt{5}) / 2 \approx 1.618$. Вершины лежат в точках вида $(0, \pm1, \pm t)$, $(\pm1, \pm t, 0)$ и $(\pm t, 0, \pm1)$. Для отрисовки используется индексный буфер: хранятся 12 уникальных вершин и 60 индексов (по 3 вершины на каждую из 20 граней).

Каждая вершина хранит позицию и процедурно вычисленный цвет:

```
struct Vertex {
   glm::vec3 position;
   glm::vec3 color;
};
```

Однородные координаты и матрицы

Обычная точка в пространстве задаётся тремя координатами (x, y, z). Для преобразований в программе к ним добавляется четвёртая компонента w = 1. Такая запись называется однородными координатами. Дополнительная компонента позволяет записать перенос как умножение на матрицу 4×4, что невозможно сделать с помощью матрицы 3×3.

В коде матрица модели строится последовательно:

1. Сдвиг (Position) — glm::translate.
2. Масштабирование (Scale) — glm::scale.
3. Повороты по осям X, Y, Z (Rotation) — glm::rotate.

Камера и проекции

Камера фиксирована в точке (0, 0, 3) и смотрит в начало координат. Матрица вида view строится с помощью функции glm::lookAt.

Итоговая матрица применяется к каждой вершине в вершинном шейдере:

gl_Position = ubo.proj * ubo.view * ubo.model * vec4(in_position, 1.0);

Перспективная проекция создаёт эффект глубины: более удалённые объекты выглядят меньше. В коде вертикальный угол обзора (FOV) задаётся ползунком в интерфейсе. Ортографическая проекция не зависит от расстояния до камеры. Обе матрицы строятся с помощью GLM (с инверсией оси Y для Vulkan). Переключатель Perspective projection выбирает нужную матрицу.

Анимация

Если включена галочка "Playing", каждый кадр угол поворота вокруг оси Y увеличивается на speed * delta_time. Также, если задан радиус траектории (Trajectory radius), объект начинает двигаться по кругу. Координаты центра вычисляются по формулам $x = r \cdot \cos \alpha$, $z = r \cdot \sin \alpha$, где α — накопительный угол.

Процедурные цвета вершин

Цвет каждой вершины задан в коде программно (процедурно) в зависимости от её позиции. Это создаёт плавный градиент (радужный эффект) благодаря интерполяции цветов видеокартой.

В интерфейсе есть ползунок Color multiplier. Выбранный цвет покомпонентно умножается на цвет вершины во фрагментном шейдере. Если поставить белый цвет — икосаэдр будет радужным. Если поставить красный — останется только красная составляющая градиента.

Объекты Vulkan и отрисовка

Координаты и цвета вершин записываются в вершинный буфер VkBuffer, а номера вершин треугольников — в индексный буфер. Uniform-буфер содержит матрицы model, view, proj и цветовой множитель, которые обновляются каждый кадр через ImGUI.

Графический конвейер VkPipeline настроен на список треугольников. Включен тест глубины, который позволяет ближним граням перекрывать дальние.

Результаты

Ниже приведены снимки работающей программы.

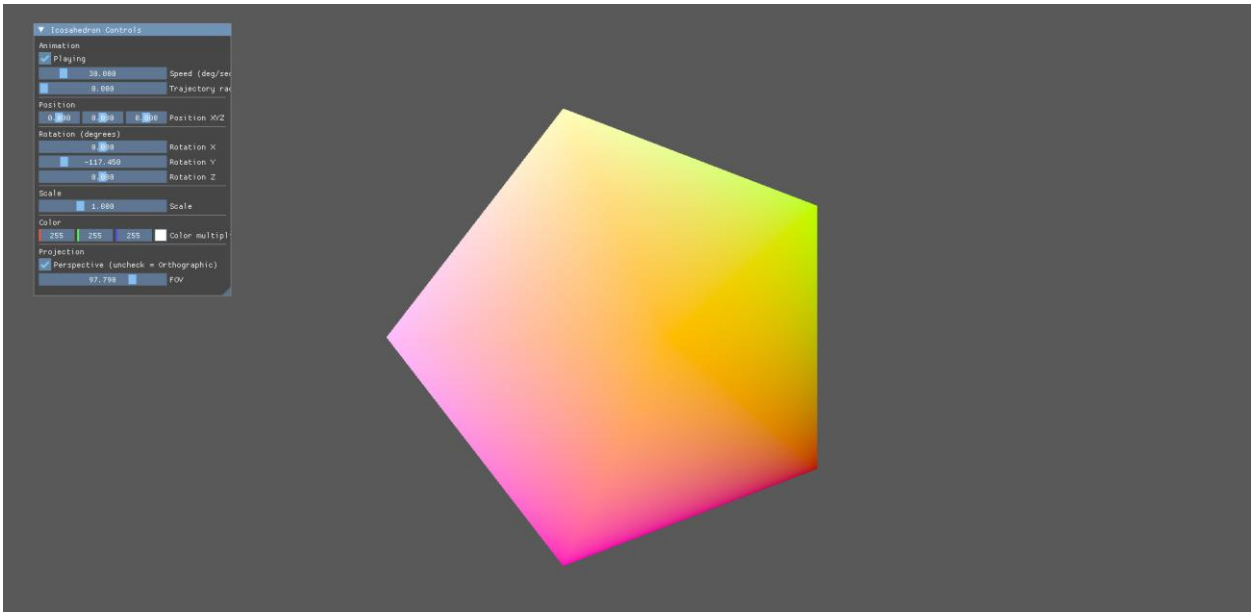


Рисунок 1 — Перспективная проекция
Икосаэдр с включенной перспективной проекцией. Виден радужный процедурный градиент и эффект уменьшения по глубине.

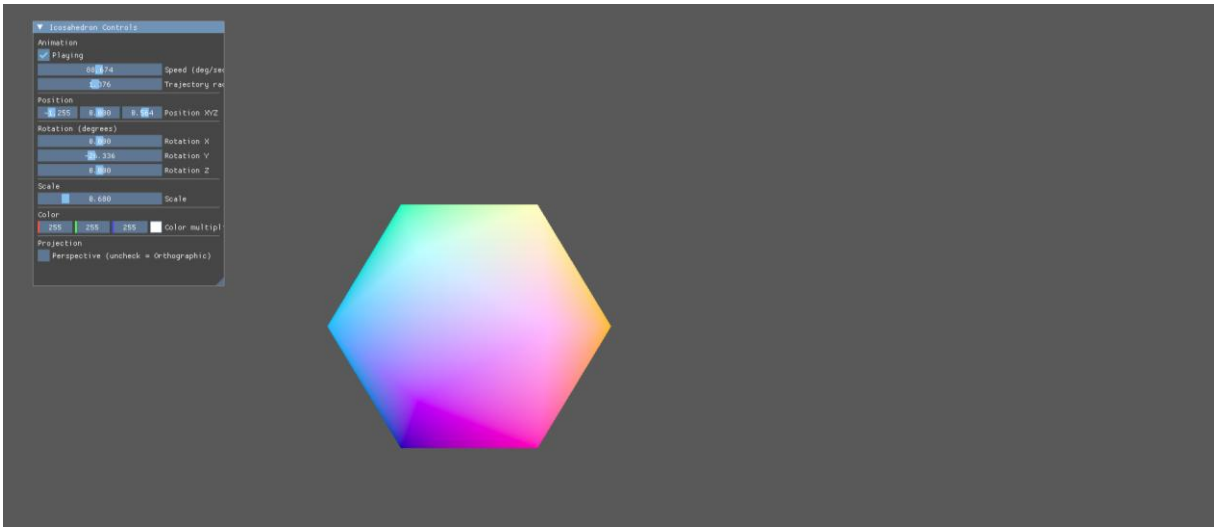


Рисунок 2 — Ортографическая проекция
Проекция переключена на ортографическую. Размер фигуры не зависит от глубины.

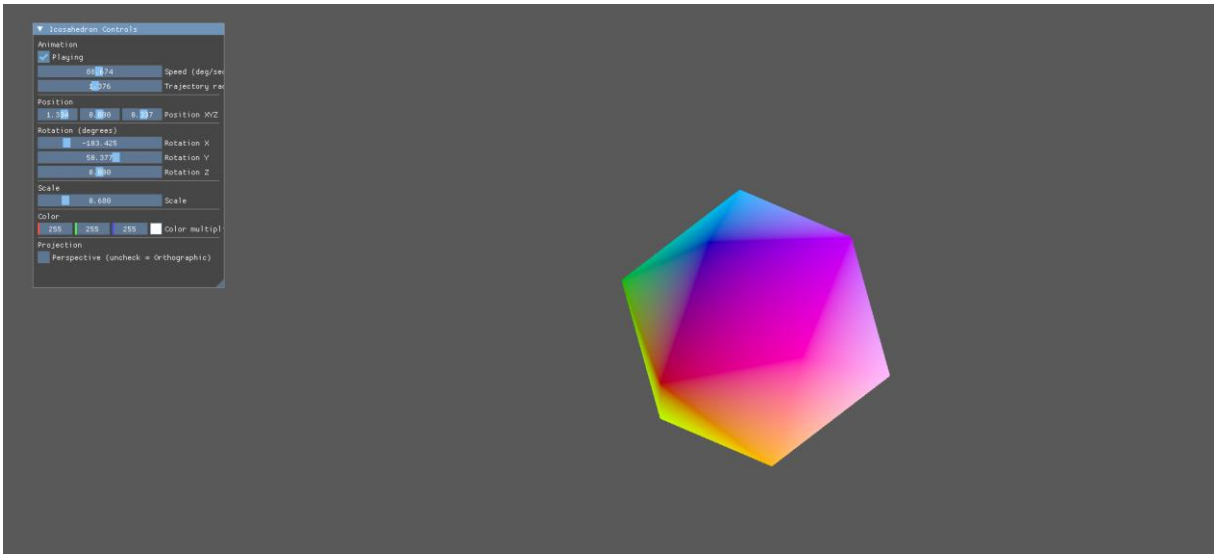


Рисунок 3 — Изменение параметров
Показан результат работы ползунка Rotation.

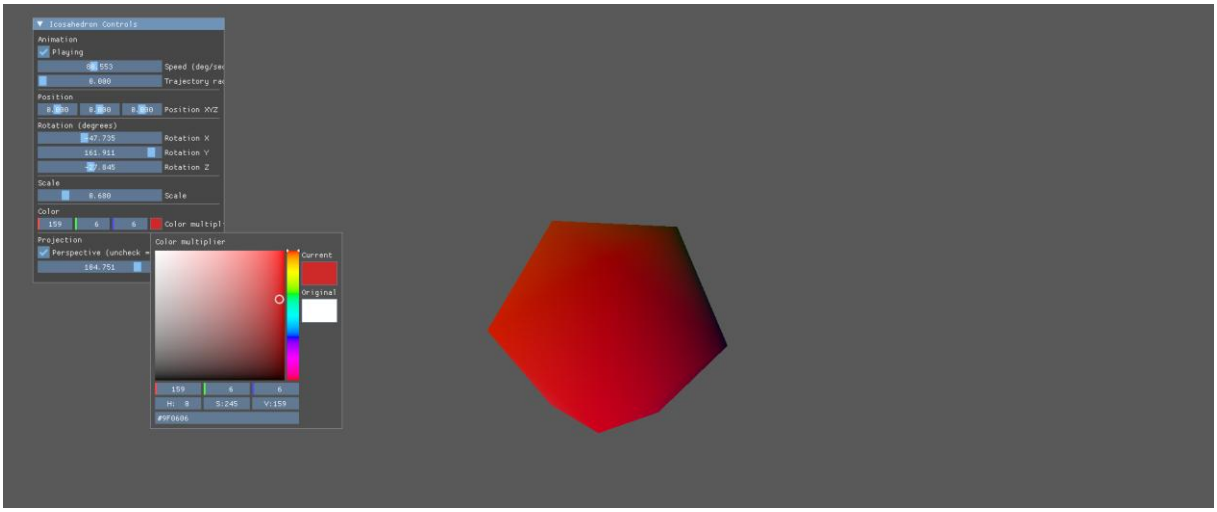


Рисунок 4 — Изменение параметров
Показан результат работы ползунка Color multiplier.

Выводы

В ходе работы я построил правильный икосаэдр и реализовал его отображение с помощью графического API Vulkan. Я изучил, как работают матрицы 4×4 и однородные координаты, а также разницу между перспективной и ортографической проекциями.

Была реализована панель управления на базе ImGui, позволяющая в реальном времени менять положение, поворот и масштаб фигуры, управлять анимацией вращения и движением по траектории. Также реализовано изменение цвета и процедурная раскраска вершин.

В результате я научился работать с вершинными и индексными буферами, передавать данные в шейдеры через uniform-буферы и настраивать графический конвейер.